



FIREBRAND

LYCEUM EDUCATION GROUP

Integrated App Project

Power Platform Pathway | Session 8

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values

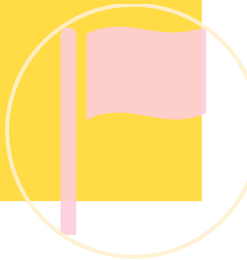


Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, **S5**, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, **K6**, **K9**, K11

Skills

S1, **S4**, **S6**, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Integrated App Project

Lesson Objectives

Create

- Create a complete app solution

Evaluate

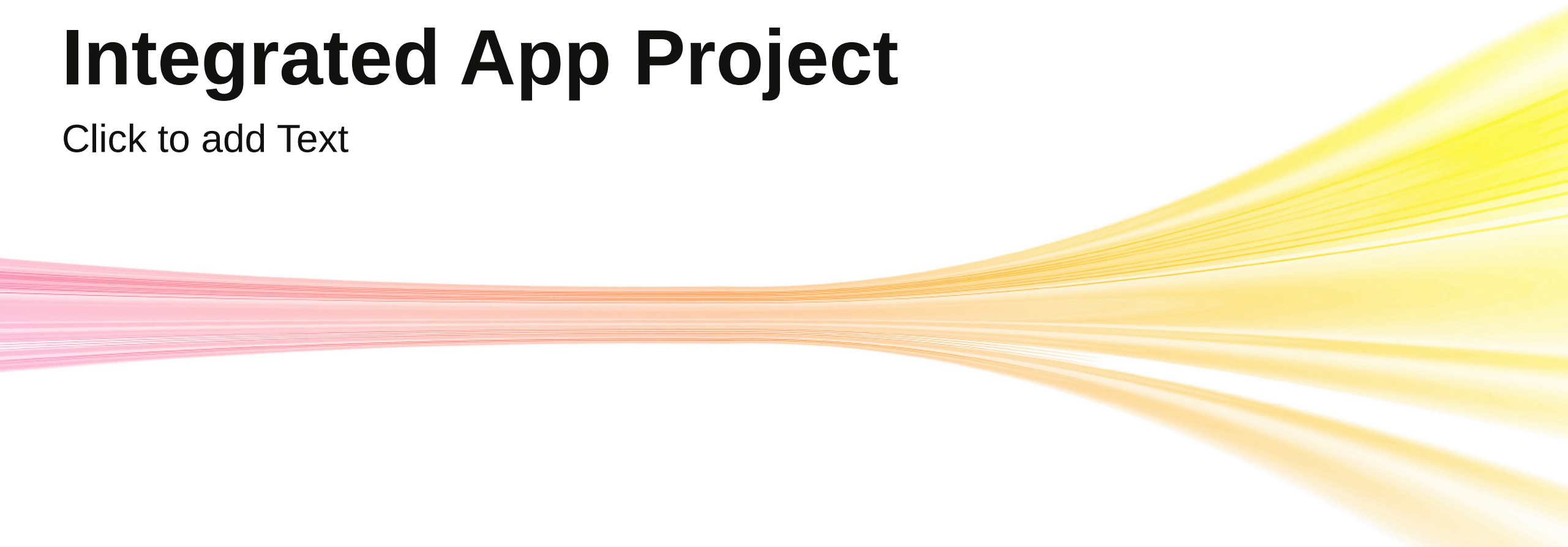
- Evaluate design and logic decisions

Apply

- Apply testing and structured development practices

Integrated App Project

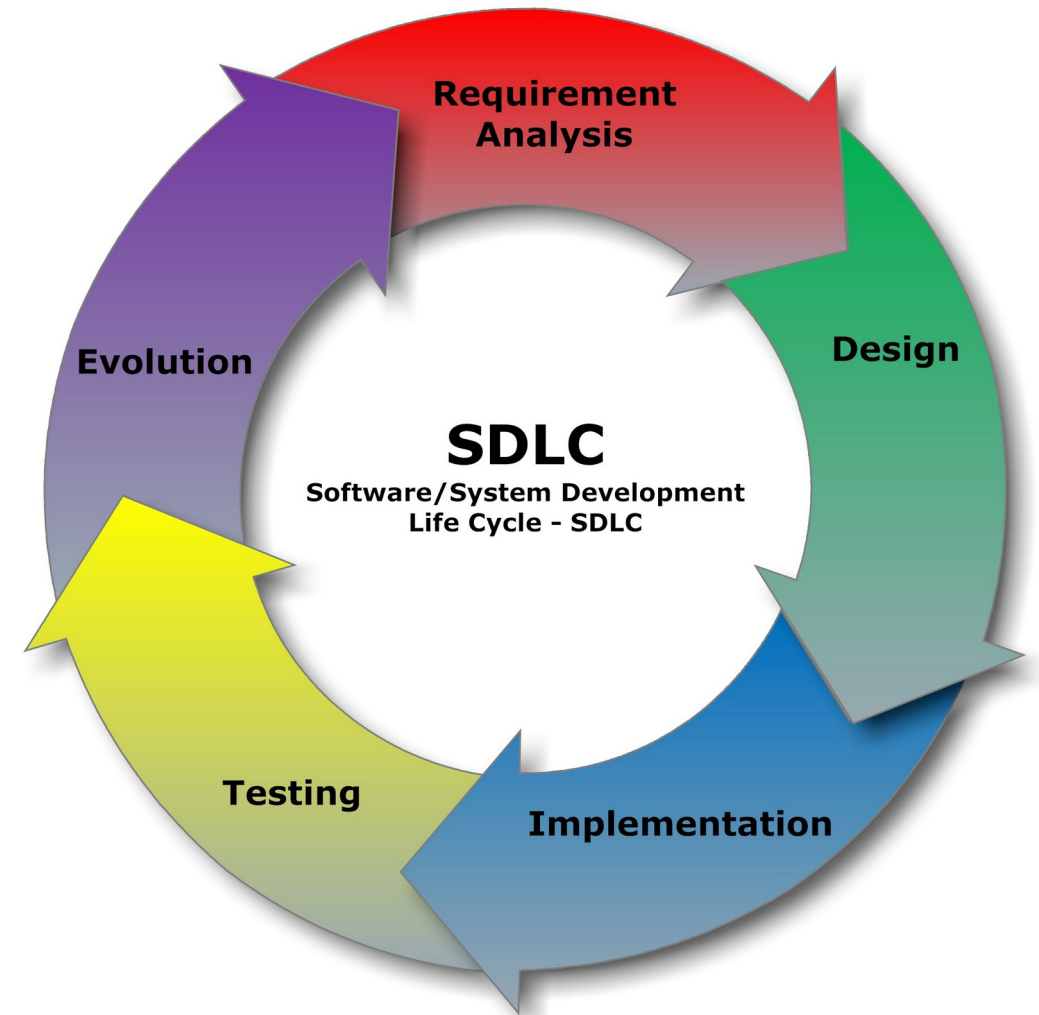
Click to add Text



Programming Essentials: Syntax, Logic & Problem Solving

- Structural planning and visual modelling
- Low-code development using collections and data sources
- Rigorous testing, peer review, and solution justification

The Big Picture: This project combines UI design, data management, logic optimisation, and quality assurance into one cohesive lifecycle.



Planning within the SDLC

- **Why Plan First?** Designing structures before writing code lowers development costs and prevents logic rewriting
- **The Defect Leakage Timeline:** Catching a structural flaw during design is up to 100 times cheaper than fixing it after deployment
- **Key Mapping Actions:**
 - **Inputs:** Identify what data users enter (forms, text controls)
 - **Outputs:** Define what users see or receive (galleries, success banners)
 - **Data Sources:** Select where data lives permanently versus temporarily



The Workplace Scenario

The Scenario:

- The internal IT Helpdesk needs a rapid-response application to log technical issues and manage hardware inventory
- **Functional Requirements:**
 - **Conditional Behaviour:** Highlight urgent tickets in red; automatically assign a 24-hour resolution deadline if urgency is 'Critical'
 - **Storage:** Store live issues in a persistent backend data source, while maintaining a local in-memory collection for active session data

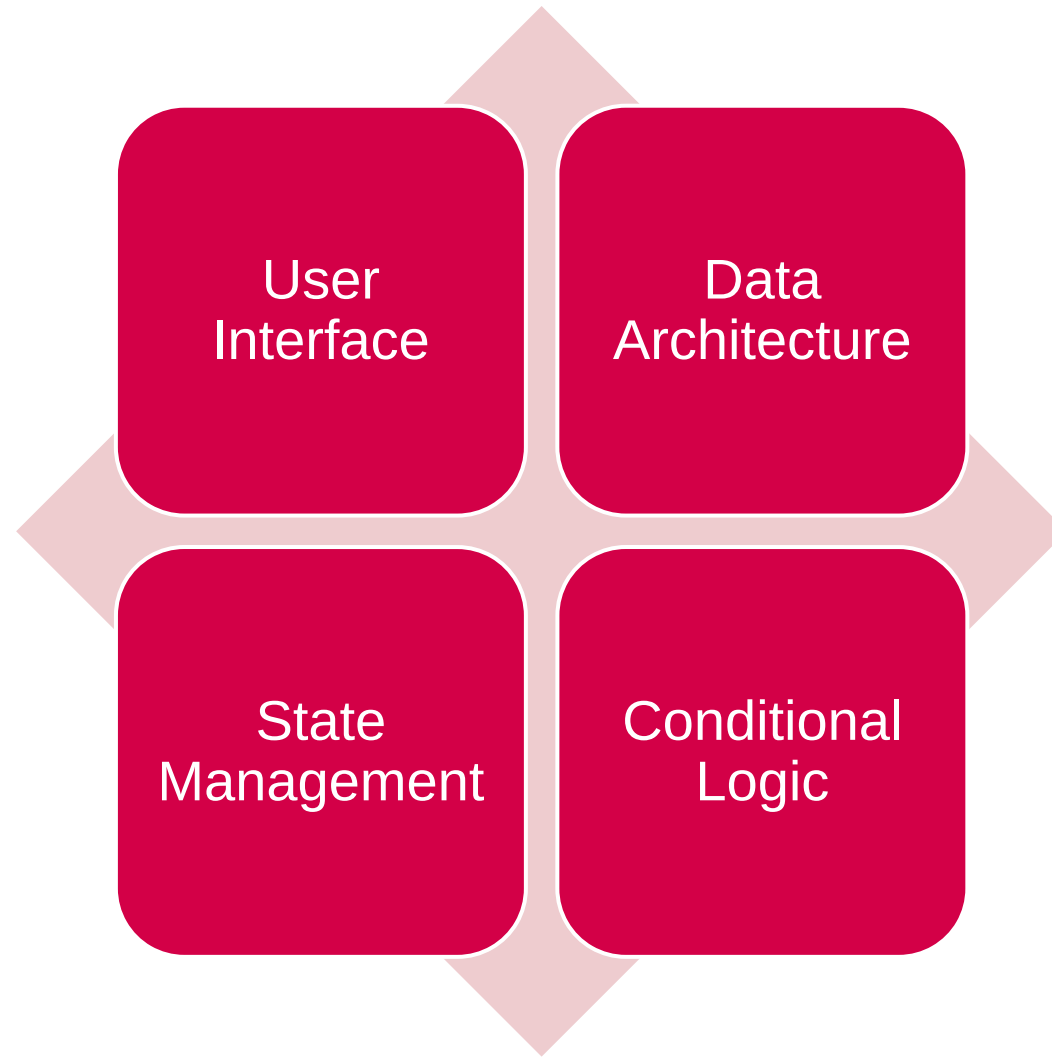
Data Intake: Capture

- user name,
- asset ID,
- issue category,
- urgency level

Functional Requirements

1. The ticket submission action control must remain locked and un-clickable (DisplayMode.Disabled) until the EmployeeName control is verified as populated via an explicit IsBlank() evaluation, and the AssetID string fulfills a mandatory validation check
2. If a user selects "Critical" from the urgency dropdown menu, the system must instantly run a Power Fx time calculation formula that populates a target resolution field exactly 24 hours into the future
3. Any row entry displayed inside a technician-facing ticket management gallery that carries a status flag of "Critical" must have its background color property altered dynamically to a soft alert red using explicit RGBA() conditional formulas

Structured Development Guidelines



Implementation within the SDLC

Core Implementation Milestones:

- Move from wireframe outlines directly into functional Power Fx expression formulas
- Run data updates via in-memory collection states rather than processing heavy, sequential transactions directly against database tables
- Implement structural validation gates right on your screen controls to catch user format errors before data is submitted



Implementation Checkpoints

Milestone 1: Functional Mapping

- Open the Variables and Collections review window. Verify that all input fields pass records cleanly without database column data type mismatches
- Inspect your backend database tables directly to confirm that test rows submit and display accurately.

Milestone 3: Formula Optimisation

- Clear all active blue wavy underlines and amber performance warning triangles from your development formula bars
- Verify that data queries use delegable statements to ensure the solution scales past the default 500-row processing ceiling

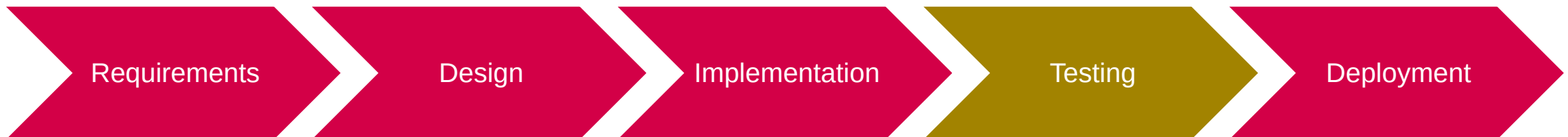
Milestone 2: UI & Navigation Polish

- Delete any duplicated static layout shapes or manual geometric elements across different screens to make the application truly modular
- Trace interactive screen navigation logic loops to confirm user movement through screens flows without hitting dead ends

Testing within the SDLC

Core Testing Methodologies:

- Isolate individual formula expressions—such as the input validation loops—to confirm they execute exactly as intended
- Verify the end-to-end data plumbing pipeline. Confirm records successfully transition from UI controls, pass through local collections, and write cleanly into backend database columns
- Intentionally stress-test your system with empty inputs, null records, and boundary values to ensure the interface handles errors safely without crashing



Rigorous Testing & Validation

	Typical User Journeys	Edge Cases	Invalid Inputs
Action Step	Fill out all data intake forms with standard alphanumeric data inputs and select the submit action	Select the urgency status control field and explicitly toggle the parameter to "Critical"	Leave the primary target required input fields blank and click the submission control trigger
Expected Application Result	Solution saves the text record to the data source table, clears all form fields, and launches a confirmation success banner	The local data engine immediately calculates an automated response deadline variable to exactly Now() + 1	The application triggers a 'Fail-Fast' validation guard, blocks data processing instantly, and shows a clear error warning

The Goal: Build a documented test history confirming your application handles errors safely without crashing

Build, Test and Present Your App



ACTIVITY



Objective: Design, build, test and present a complete workplace app

Tasks / Requirements:

- Capture data through user input forms
- Store and retrieve data using collections or a connected data source
- Implement conditional logic to control behaviour and user experience
- Display information using galleries, forms and other components

Key Point: Be ready to justify your use of data sources, components and logic

Extension Activity: Add advanced features or additional user interactions beyond the minimum



ACTIVITY



Collaborative Peer Review

Peer Review Instructions:

- **Station Swap Exchange:** Cross-examine your assigned partner's current staging deployment using three developer-focused quality metrics
- **Constructive Feedback Loops:** Identify clear opportunities for logical simplification or backend performance tuning rather than giving generic design summaries
- **Refinement Integration:** Formally log your colleague's feedback directly into your validation notes and refactor your app controls before final presentation delivery

Expected Lab Outcomes:

- Evidence of intentional, user-focused interface adjustments
- Documentation tracking alternate code structures considered during development
- An optimised, verified application that successfully passes mutual peer assurance

Review Matrix

Evaluation Target	Peer Quality Checklist Focus Vector	Target EPA Alignment
Usability & UX	Are screen flows easy to navigate, and do fields use dynamic labels instead of confusing error text?	S2 (Responsive Designs)
Code Structure	Are structural controls named cleanly, or is there confusing variable clutter across screens?	K1, K7 (Clean Code)
Formula Logic	Are calculations built using flat execution commands, or does the code rely on messy nested formulas?	S1 (Logic Synthesis)

Presentation & Solution Justification



The Pitch: Present your complete app solution and justify your architectural decisions

Key Presentation Points:

- **Requirements Match:** Demonstrate exactly how the app solves the IT tracker problem
- **Data & Design Justification:** Explain why you chose specific data structures or custom components
- **Code Maintainability:** Explain your approach to keeping formulas clean and readable for future developers
- **Self-Evaluation:** Identify the core strengths of your app and list areas you would expand in the next phase

Presentations



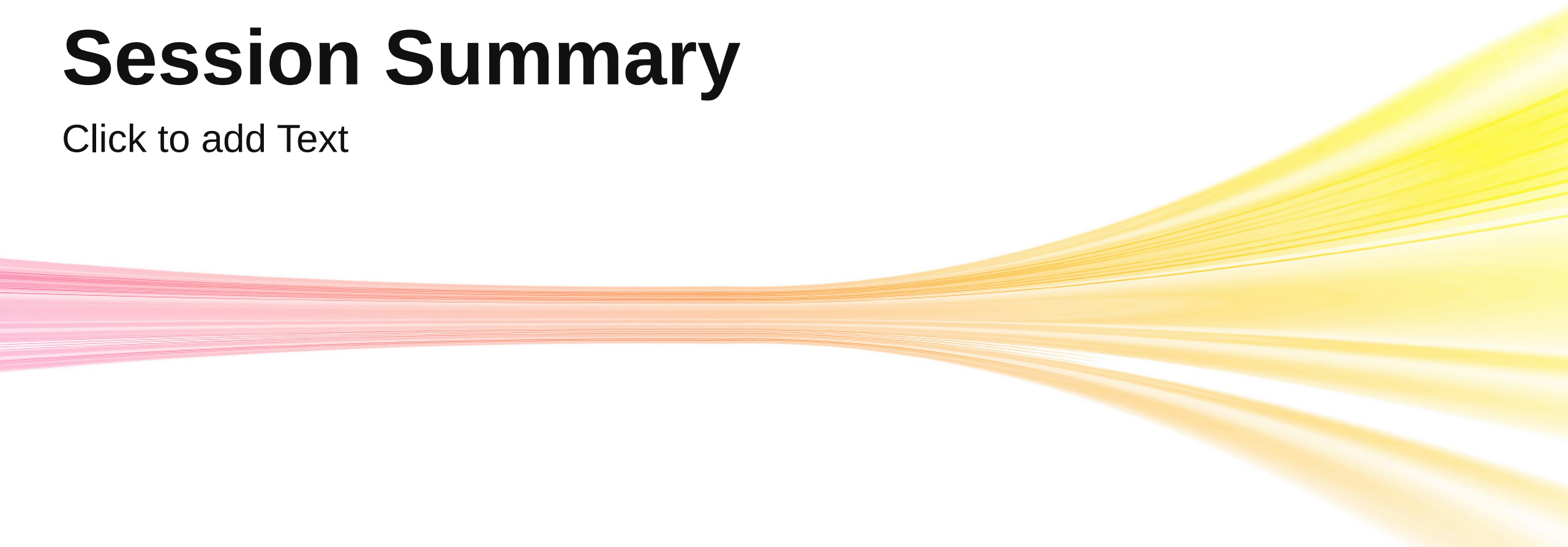
Stretch and Challenge

The Mission: Create an enhanced solution demonstrating independent problem-solving and application of advanced Power Apps development techniques

- **Task 1:** Extend the app functionality beyond the minimum requirements
- **Task 2:** Improve the app performance through optimisation techniques
- **Task 3:** Add advanced features or additional user interactions to the app
- **Task 4:** Justify your design decisions with reference to usability, maintainability and efficiency

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Integrated App Project

Lesson Objectives

Create

- Create a complete app solution

Evaluate

- Evaluate design and logic decisions

Apply

- Apply testing and structured development practices

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, **S5**, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, **K6**, **K9**, K11

Skills

S1, **S4**, **S6**, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

